Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen VII: Monografías de SQL, Shell, PHP, Ruby, Dart, Funcionales y Científicos

Parte V (Bloques A–H): Consultas declarativas, automatización UNIX, ecosistemas web/móvil, programación funcional pura, BEAM y cálculo científico

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Monografías de Datos, Automatización de Sistemas y Desarrollo Web/Móvil 1

1. Monografía de SQL (Bloques A−H): Álgebra Relacional, DDL/DML, Lógica Trivaluada, CTEs Recursivas, Funciones de Ventana y Optimización 2
1.1. Bloques A−D: Modelo relacional, DDL con restricciones, orden lógico de SELECT, lógica trivaluada (NULL) y JOINs . . . 2
1.2. Bloques E−H: CTEs recursivas, funciones de ventana (OVER), transacciones, EXPLAIN y projects/sql-transactional-ledger . . . 3

2. Monografía de Bash y POSIX Shell (Bloques A−H): Tuberías, Redirecciones, Expansiones Seguras, Trap, Herramientas UNIX y Proyectos 5
2.1. Bloques A−D: Filosofía UNIX, POSIX sh vs Bash, descriptores de fichero, redirecciones, entrecomillado y expansiones . . . 5
2.2. Bloques E−H: Modo estricto, arrays, trap de limpieza, herramientas (awk, sed, jq), ShellCheck y 4 proyectos . . . 5

3. Monografías de PHP 8.3+, Ruby 3.3+ y Dart 3.x (Bloques A−H): Motores Zend, YJIT y Compilación AOT/JIT Multiplataforma 8
3.1. Monografía de PHP 8.3+ (Bloques A−H): Zend Engine, tipado estricto, enums, Composer/PSR y arquitectura web . . . 8
3.2. Monografías de Ruby 3.3+ y Dart 3.x (Bloques A−H): Bloques/Metaprogramación en Ruby e Isolates/AOT en Dart . . . 8

II Monografías de Lenguajes Funcionales Estáticos y Concurrentes (Haskell, OCaml, F#, Scala, BEAM, Lisp) 11

4. Monografía de Haskell (Bloques A−H): Pureza, Evaluación Perezosa, Inferencia Hindley-Milner, Typeclasses y Mónadas 12
4.1. Bloques A−D: Filosofía pura de Haskell, GHC/Cabal/Stack, currificación, evaluación perezosa, ADTs y Typeclasses . . . 12
4.2. Bloques E−H: Functor, Applicative, Monad (IO, Reader, State, Either), concurrencia STM, QuickCheck y proyectos . . . 12

5. Monografías de OCaml 5.x y F# (.NET) (Bloques A−H): Módulos de Orden Superior (Functors), Efectos Algebraicos y Expresiones de Computación 15
5.1. Monografía de OCaml 5.x (Bloques A−H): Inferencia ML, Functors de módulos, representación en memoria y efectos algebraicos . . . 15
5.2. Monografía de F# sobre .NET (Bloques A−H): Tuberías, unidades de medida ([<Measure>]), Computation Expressions y proyectos . . . 16

6. Monografías de Elixir, Erlang/OTP (BEAM), Scala 3, Clojure y Lisp/Scheme (Bloques A−H) 18
6.1. Monografías de Erlang/OTP y Elixir sobre la BEAM (Bloques A−H): Actores aislados, GenServer y Supervisores . . . 18
6.2. Monografías de Scala 3, Clojure y Common Lisp / Scheme (Bloques A−H): Tipos dependientes, HAMT y macros . . . 19

III Monografías de Lenguajes Científicos, Embebidos y Lógicos (Julia, R, Lua, Perl, Objective-C, Prolog, GDScript) 21

7. Monografías de Julia y R (Bloques A–H): Despacho Múltiple, Especialización JIT LLVM, Vectorización y Ciencia de Datos 22
7.1. Monografía de Julia (Bloques A–H): Despacho múltiple, estabilidad de tipos, broadcasting fusionado y HPC . . . 22
7.2. Monografía de R (Bloques A–H): Vectores, data frames, fórmulas estadísticas, Tidyverse/data.table y Quarto . . . 23

8. Monografías de Lua 5.4+, Perl 5, Objective-C, Prolog y GDScript (Bloques A–H): Scripting Embebido, Unificación Lógica y Motores 25
8.1. Monografías de Lua 5.4+, Perl 5 y Objective-C (Bloques A–H): Tablas/Metatablas, Regex y objc_msgSend . . . 25
8.2. Monografías de Prolog y GDScript (Bloques A–H): Unificación lógica (SLD) y programación de motores de videojuegos . . . 26

A. Notación 28

B. Hoja de fórmulas 29

C. Soluciones 30
C.1. Monografía de SQL (Bloques A–H): Álgebra Relacional, DDL/DML, Lógica Trivaluada, CTEs Recursivas, Funciones de Ventana y Optimización . . . 30
C.2. Monografía de Bash y POSIX Shell (Bloques A–H): Tuberías, Redirecciones, Expansiones Seguras, Trap, Herramientas UNIX y Proyectos . . . 30
C.3. Monografías de PHP 8.3+, Ruby 3.3+ y Dart 3.x (Bloques A–H): Motores Zend, YJIT y Compilación AOT/JIT Multiplataforma . . . 31
C.4. Monografía de Haskell (Bloques A–H): Pureza, Evaluación Perezosa, Inferencia Hindley-Milner, Typeclasses y Mónadas . . . 32
C.5. Monografías de OCaml 5.x y F# (.NET) (Bloques A–H): Módulos de Orden Superior (Functors), Efectos Algebraicos y Expresiones de Computación . . . 32
C.6. Monografías de Elixir, Erlang/OTP (BEAM), Scala 3, Clojure y Lisp/Scheme (Bloques A–H) . . . 33
C.7. Monografías de Julia y R (Bloques A–H): Despacho Múltiple, Especialización JIT LLVM, Vectorización y Ciencia de Datos . . . 34
C.8. Monografías de Lua 5.4+, Perl 5, Objective-C, Prolog y GDScript (Bloques A–H): Scripting Embebido, Unificación Lógica y Motores . . . 34

D. Auditoría de fuentes 36

E. Plan de estudio 38

Parte I

Monografías de Datos, Automatización de Sistemas y Desarrollo Web/Móvil

Capítulo 1

Monografía de SQL (Bloques A–H): Álgebra Relacional, DDL/DML, Lógica Trivaluada, CTEs Recursivas, Funciones de Ventana y Optimización

*Este capítulo desarrolla la monografía completa (Bloques A – H) de **SQL (ISO/IEC 9075:2023)**. SQL no es un simple lenguaje de consulta superficial, sino un lenguaje declarativo funcional basado en el álgebra y el cálculo relacional de primer orden (con recursión de punto fijo y funciones analíticas de ventana) que todo ingeniero de software y datos debe dominar en profundidad.*

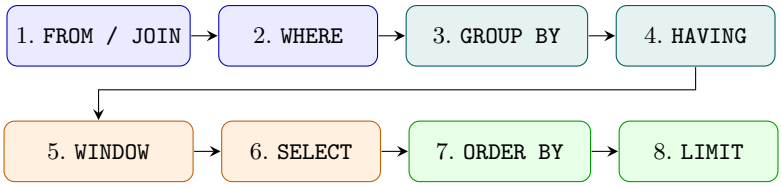


Figura 1. Orden lógico de evaluación de una consulta SQL frente al orden léxico de escritura: un alias definido en SELECT no existe aún cuando se evalúan WHERE, GROUP BY o HAVING.

Consejo

En funciones de ventana con `ORDER BY` que calculan acumulados (`SUM(...) OVER (ORDER BY fecha)`), especifica explícitamente `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`: el marco por defecto del estándar SQL cuando hay `ORDER BY` es `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, que agrupa de golpe todas las filas empatadas en `fecha`.

Definición

Predicado Sargable en SQL (Search Argument Able): condición de filtro de la forma `columna_indexada OP expresión_constante` que permite al motor de almacenamiento posicionarse directamente en las páginas hoja de un árbol B+ (`Index Seek / Index Range Scan`) sin evaluar una transformación fila por fila.

Autoficha

¿Qué diferencia exacta de numeración producen `ROW_NUMBER()`, `RANK()` y `DENSE_RANK()` ante una secuencia de puntuaciones `[100, 90, 90, 80]`?

1.1 Bloques A–D: Modelo relacional, DDL con restricciones, orden lógico de SELECT, lógica trivaluada (NULL) y JOINs

Aunque en SQL escribimos léxicamente `SELECT ... FROM ... WHERE ...`, el motor relacional evalúa la consulta siguiendo el orden lógico estricto mostrado en la Figura \ref{fig:fig-sql-logical-execution-order}: `FROM` y `JOIN` construyen primero el producto cartesiano filtrado por `ON`; `WHERE` descarta las filas cuya condición no evalúa a `TRUE`; `GROUP BY` particiona las filas en grupos; `HAVING` filtra grupos agregados; `WINDOW` calcula funciones de ventana; `SELECT` proyecta las columnas y asigna alias; `DISTINCT` elimina duplicados; `ORDER BY` ordena el conjunto resultante; y `LIMIT/FETCH FIRST` recorta la salida. Un escollo clásico proviene de la **lógica trivaluada (3VL)** de `NULL` (`TRUE`, `FALSE`, `UNKNOWN`): cualquier comparación ordinaria `col = NULL` o `col <> NULL` produce `UNKNOWN`. Si una subconsulta de `WHERE id NOT IN`

`(SELECT parent_id FROM t)` devuelve un solo `NULL`, la expresión `id <> v1 AND id <> NULL` evalúa a `UNKNOWN` para todas las filas y la consulta devuelve un conjunto vacío; por ello debe utilizarse siempre `WHERE NOT EXISTS (...)` o `IS DISTINCT FROM`.

1.2 Bloques E–H: CTEs recursivas, funciones de ventana (OVER), transacciones, EXPLAIN y projects/sql-transactional-ledger

En el Nivel Avanzado y Experto (Bloques E y F), las **expresiones de tabla común recursivas** (`WITH RECURSIVE`) permiten recorrer árboles jerárquicos y grafos (como el plan contable de nuestro proyecto verificado `projects/sql-transactional-ledger`), mientras que las **funciones de ventana** (`OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`) calculan saldos acumulados, medias móviles y clasificaciones (`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG`, `LEAD`) en una sola pasada sin colapsar las filas individuales:

```
SELECT
    account_id,
    entry_id,
    debit_cents - credit_cents AS delta_cents,
    SUM(debit_cents - credit_cents) OVER (
        PARTITION BY account_id
        ORDER BY entry_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_balance_cents
FROM journal_lines;
```

Para que el optimizador de costes pueda emplear un índice B+ Tree, los predicados de `WHERE` deben ser **sargables** (*Search Argument Able*): aplicar una función sobre la columna indexada (`WHERE YEAR(created_at) = 2026`) impide el recorrido por rango del índice y fuerza un escaneo completo de tabla (`Seq Scan`), debiendo reescribirse como `WHERE created_at >= '2026-01-01' AND created_at < '2027-01-01'`.

Problema resuelto

Enunciado En el proyecto `projects/sql-transactional-ledger`, la cuenta de Caja (`account_id = 110`) registra tres apuntes sucesivos `(debit_cents, credit_cents)`: `(500000, 0)`, `(125000, 0)` y `(0, 40000)`. Calcula el saldo acumulado en céntimos (`running_balance_cents`) que devuelve la función de ventana `SUM(debit_cents - credit_cents) OVER (...)` en la tercera fila.

Datos

- $d_1 = 500000$ centimos — Aportación inicial de capital
- $d_2 = 125000$ centimos — Cobro de factura SaaS
- $c_3 = 40000$ centimos — Pago de infraestructura cloud

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Cálculo de saldo acumulado por función de ventana SQL en sql-transactional-ledger

$$B_3 = 500000 + 125000 - 40000$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos los tres deltas algebraicos de la partición account$_i d = 110$

$$B_3 = 500000 + 125000 - 40000$$

$$(((500000 - 0) + (125000 - 0)) + (0 - 40000))$$

Resultado

$$Resultado = 585000$$

Interpretación El saldo acumulado en la tercera fila es exactamente 585 000 céntimos (5850.00 EUR), idéntico a la aserción de `verify_ledger.py`.
Comprobación de los pasos

- Paso 1: 585000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) Ante las puntuaciones `[100, 90, 90, 80]`, ¿qué valor entero asigna `RANK()` al cuarto elemento (`80`) tras el empate en la posición 2?
2 [PRB-0004] (Básico, Numérico) Ante las mismas puntuaciones `[100, 90, 90, 80]`, ¿qué valor entero asigna `DENSE_RANK()` al cuarto elemento (`80`)?
3 [PRB-0005] (Intermedio, Desarrollo por pasos) Un producto cartesiano `CROSS JOIN` entre una tabla de 120 tiendas y una tabla de 365 días del año genera todas las combinaciones `(tienda, día)`. ¿Cuántas filas produce?
4 [PRB-0006] (Intermedio, Diseño) Una consulta con paginación `LIMIT 50 OFFSET 400` devuelve la página 9 de resultados. ¿Cuál es el número de orden (base 1) de la primera fila devuelta en esa página ($400 + 1$)?

Capítulo 2

Monografía de Bash y POSIX Shell (Bloques A–H): Tuberías, Redirecciones, Expansiones Seguras, Trap, Herramientas UNIX y Proyectos

*Este capítulo presenta la monografía completa (Bloques $A-H$) de **Bash y POSIX Shell**. Los scripts de shell son el pegamento universal de los sistemas Linux/macOS, los contenedores Docker y los pipelines de CI/CD. Enseñamos a escribirlos con rigor de ingeniería de sistemas: inmunes a espacios en nombres de fichero, con limpieza atómica de directorios temporales y auditados con* `shellcheck`.

Consejo

Integra siempre `shellcheck -S warning script.sh` en tu editor y en CI: detecta al instante variables sin entrecomillar (SC2086), pérdida de estado de salida al usar `local x=$(cmd)` en una sola línea (SC2155) y problemas de subshells en tuberías.

Definición

Código de estado de salida por señal en POSIX (`128 + N`): convención del intérprete de comandos según la cual un proceso terminado de forma abrupta por la señal número N reporta en `$?` el código de salida $128 + N$ (por ejemplo, `130` para `SIGINT = 2` o `137` para `SIGKILL = 9`).

Autoficha

¿Por qué la línea `local salida="$(comando_que_falla)"` en Bash oculta el código de error de `comando_que_falla` (porque `local` es un comando builtin cuyo propio código de retorno es `0`) y debe escribirse en dos líneas separadas (`local salida; salida="$(comando_que_falla)"`)?

2.1 Bloques A–D: Filosofía UNIX, POSIX sh vs Bash, descriptores de fichero, redirecciones, entrecomillado y expansiones

En el modelo de procesos POSIX, cada comando hereda tres descriptores de fichero abiertos: `0` (entrada estándar), `1` (salida estándar) y `2` (error estándar), y al terminar devuelve un código de estado entero entre `0` y `255` donde **`0` significa éxito** (verdadero en `if cmd; then` y en los operadores de cortocircuito `cmd1 && cmd2` y `cmd1 || cmd2`) y cualquier valor distinto de cero indica fallo. El orden de las redirecciones se evalúa de izquierda a derecha: `cmd > salida.log 2>&1` apunta primero el descriptor `1` a `salida.log` y luego duplica el descriptor `2` hacia donde apunta `1` (ambos van al fichero). La causa número uno de fallos y vulnerabilidades de inyección en scripts de shell es dejar expansiones sin comillas dobles (`$archivo` en lugar de `"$archivo"`), lo que dispara la división por palabras (*word splitting* según `IFS`) y la expansión de comodines (*pathname expansion* `*`, `?`).

2.2 Bloques E–H: Modo estricto, arrays, trap de limpieza, herramientas (awk, sed, jq), ShellCheck y 4 proyectos

Todo script de automatización en Bash debe comenzar configurando el modo estricto (`set -euo pipefail`, que aborta ante comandos fallidos, variables no definidas y errores intermedios en una tubería) y registrando un manejador `trap` sobre la pseudo-señal `EXIT` para eliminar de forma garantizada los recursos temporales:

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'

readonly WORK_DIR="$(mktemp -d)"
cleanup() {
    rm -rf -- "$WORK_DIR"
}
trap cleanup EXIT INT TERM

count_non_empty_lines() {
    local target_file="${1:?Ruta de fichero obligatoria}"
    grep -cve '^[[:space:]]*$' -- "$target_file"
}
```

Observa el uso de `-` antes de `"$target_file"` y `"$WORK_DIR"`: indica el fin de las opciones de bandera, impidiendo que un fichero cuyo nombre empiece por guion (`-rf`) sea interpretado como una opción por `grep` o `rm`. Para procesar listas de ficheros con nombres arbitrarios (incluyendo saltos de línea), se combina siempre el delimitador nulo: `find . -type f -name '*.log' -print0 | xargs -0 sha256sum`.

Problema resuelto

Enunciado Un contenedor en un pipeline de despliegue termina abruptamente con código de salida `$? = 137` tras intervenir el *OOM Killer* del kernel Linux (`SIGKILL = 9`), y otro proceso aborta por violación de segmento (`SIGSEGV = 11`, código $128+11=139$). Calcula la suma de los números de las dos señales POSIX subyacentes obtenidas restando 128 a cada código de salida $((137-128)+(139-128))$.

Datos

- $E_1 = 137$ — Código de salida por SIGKILL (128 + 9)
- $E_2 = 139$ — Código de salida por SIGSEGV (128 + 11)

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Decodificación de señales POSIX a partir del código de estado 128 + N

$$S = (137-128)+(139-128)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos 128 a cada código para recuperar los números de señal 9 y 11 y los sumamos

$$S = (137-128)+(139-128)$$

$$((137-128)+(139-128))$$

Resultado

$$\mathit{Resultado} = 20$$

Interpretación Las señales subyacentes son la 9 (SIGKILL) y la 11 (SIGSEGV), cuya suma es 20.

Comprobación de los pasos

- Paso 1: 20 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0009] (Básico, Numérico) Un script en ejecución es interrumpido por el usuario pulsando `Ctrl+C` (`SIGINT = 2`). ¿Qué código de salida `$?` reporta el shell ($128 + 2$)?
2 [PRB-0010] (Básico, Numérico) Un proceso recibe la señal de terminación ordenada `SIGTERM = 15` sin capturarla. ¿Cuál es su código de salida ($128 + 15$)?
3 [PRB-0011] (Intermedio, Desarrollo por pasos) Un comando `xargs -0 -n 50 -P 4` procesa una lista de 600 ficheros pasándolos en lotes de 50 ficheros por invocación. ¿Cuántos procesos hijos lanza en total `xargs` ($600 / 50$)?
4 [PRB-0012] (Intermedio, Diseño) Una máscara `umask 027` se resta de los permisos base `666` (octal) para la creación de ficheros regulares ($666 - 27$ en dígito a dígito: $6-0=6$, $6-2=4$, $6-7 \to 0$). ¿Cuál es el permiso octal resultante expresado como entero de tres dígitos?

Capítulo 3

Monografías de PHP 8.3+, Ruby 3.3+ y Dart 3.x (Bloques A–H): Motores Zend, YJIT y Compilación AOT/JIT Multiplataforma

*Este capítulo agrupa las monografías completas (Bloques A – H) de tres lenguajes esenciales en el desarrollo web y móvil de producción: **PHP 8.3+**, **Ruby 3.3+** y **Dart 3.x**.*

Consejo

En PHP 8+, incluye siempre `declare(strict_types=1);` en la primera línea de todos tus ficheros `.php` y utiliza la expresión `match ($valor) { ... }` en lugar de `switch ($valor)`: `match` compara con identidad estricta (`===`), no requiere `break` y lanza `UnhandledMatchError` si olvidas un caso.

Definición

Nulabilidad Sólida (Sound Null-Safety en Dart 3): propiedad del sistema de tipos según la cual, una vez que el programa compila, es matemáticamente imposible en tiempo de ejecución que una expresión de tipo no anulable `T` evalúe a `null`, permitiendo al compilador AOT eliminar comprobaciones de puntero nulo.

Autoficha

¿Qué diferencia exacta existe en Ruby entre una instancia de `Proc.new { return 1 }` y un `lambda` (`-> { return 1 }`) en cuanto al efecto de `return` y la comprobación del número de argumentos?

3.1 Monografía de PHP 8.3+ (Bloques A–H): Zend Engine, tipado estricto, enums, Composer/PSR y arquitectura web

PHP 8.x ha experimentado una transformación profunda respecto a sus versiones históricas: al encabezar cada fichero con `declare(strict_types=1);`, el motor Zend desactiva las coerciones escalares silenciosas y verifica contratos de tipos de unión (`int|float`), intersección (`Countable&Iterator`), clases inmutables `readonly class`, enumeraciones respaldadas (`enum Status: string`), atributos de metadatos (`#[Route(...)]`) y expresiones `match` con comparación estricta (`===`). En producción, **OPcache** mantiene el bytecode precompilado en memoria compartida entre trabajadores PHP-FPM (o en servidores persistentes como FrankenPHP / RoadRunner), y el gestor de paquetes `Composer` junto con las normas **PSR** (*PHP Standards Recommendations*: PSR-4 autoloading, PSR-7 mensajes HTTP, PSR-11 contenedores DI, PSR-15 middlewares) articula los ecosistemas de `Symfony` y `Laravel`, verificados estáticamente con `PHPStan` (nivel máximo) y probados con `PHPUnit`.

3.2 Monografías de Ruby 3.3+ y Dart 3.x (Bloques A–H): Bloques/Metaprogramación en Ruby e Isolates/AOT en Dart

En **Ruby 3.3+** (creado por Yukihiro «Matz» Matsumoto), absolutamente todo valor es un objeto y toda operación es un envío de mensaje; las clases son abiertas, la herencia múltiple se resuelve linealmente mediante *mixins* (`include` e `prepend` de módulos) y cada método puede recibir un bloque anónimo invocado con `yield` o convertido en clausura `Proc`. Con el compilador JIT en proceso **YJIT** (escrito en Rust dentro de CRuby), las aplicaciones `Ruby on Rails` logran mejoras notables de latencia, mientras

que los **Ractors** proporcionan paralelismo de actores sin compartir estado mutable. Por su parte, **Dart 3.x** ofrece **nulabilidad sólida** (*sound null-safety*: el compilador AOT garantiza que una variable no anulable jamás contiene `null`, optimizando el código máquina emitido), clases selladas (`sealed class`) con *pattern matching* exhaustivo y un modelo de concurrencia basado en **Isolates** (hebras con montículos de memoria independientes que se comunican por puertos de mensajes o transferencia de `TransferableTypedData`), compilando en modo JIT con *stateful hot reload* durante el desarrollo y en binario nativo **AOT** para producción en iOS, Android, escritorio y web:

```
sealed class ApiResponse<T> {}
final class ApiSuccess<T> extends ApiResponse<T> {
  final T data;
  const ApiSuccess(this.data);
}
final class ApiFailure<T> extends ApiResponse<T> {
  final int statusCode;
  const ApiFailure(this.statusCode);
}

String describeResponse<T>(ApiResponse<T> res) => switch (res) {
  ApiSuccess(data: final d)       => 'OK: $d',
  ApiFailure(statusCode: final c) => 'HTTP Error $c',
};
```

Problema resuelto

Enunciado Un servidor de producción con 8192 MiB de RAM reserva 2048 MiB para el sistema operativo y el búfer de OPcache/Redis, dejando $M_{\text{avail}} = 6144$ MiB para el pool de trabajadores PHP-FPM. Si cada trabajador consume en carga punta $m_{\text{worker}} = 48$ MiB de memoria privada, calcula el valor óptimo de `pm.max_children` ($6144 / 48$).

Datos

- $M_{avail} = 6144\,\text{MiB}$ — RAM neta disponible para trabajadores PHP-FPM
- $m_{worker} = 48\,\text{MiB}$ — Consumo privado RSS por proceso trabajador

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Dimensionamiento de pm.max_children en PHP-FPM sin agotamiento de RAM

$$N_{\text{máx}} = \frac{6144}{48}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Dividimos la memoria disponible entre el consumo por trabajador

$$N_{\text{máx}} = \frac{6144}{48}$$

$$\frac{6144}{48}$$

Resultado

$$Resultado = 128$$

Interpretación El servidor puede ejecutar de forma segura hasta 128 trabajadores PHP-FPM concurrentes.

Comprobación de los pasos

- Paso 1: 128 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0015] (Básico, Numérico) En Flutter/Dart, para sostener una tasa de refresco de pantalla de 120 Hz, el presupuesto total por fotograma (construcción + diseño + pintado) es 1000/120 ms. ¿Cuántos milisegundos caben en 12 fotogramas consecutivos?
2 [PRB-0016] (Básico, Numérico) En Ruby, la expresión `(1..10).select(&:even?).map { |x| x * 3 }.sum` suma el triple de los pares del 2 al 10 ($(2+4+6+8+10)\times 3$). ¿Cuál es el resultado?
3 [PRB-0017] (Intermedio, Desarrollo por pasos) Si OPcache evita recompilar 85 ficheros PHP por petición HTTP (ahorrando 0,4 ms por fichero), ¿cuántos milisegundos de CPU ahorra en cada petición ($85 \times 0{,}4$)?
4 [PRB-0018] (Intermedio, Diseño) Un Isolate de fondo en Dart procesa un lote de 400 imágenes en grupos de 25 imágenes devolviendo el progreso al Isolate principal tras cada grupo. ¿Cuántos mensajes de progreso envía?

Parte II

Monografías de Lenguajes Funcionales Estáticos y Concurrentes (Haskell, OCaml, F#, Scala, BEAM, Lisp)

Capítulo 4

Monografía de Haskell (Bloques A–H): Pureza, Evaluación Perezosa, Inferencia Hindley-Milner, Typeclasses y Mónadas

*Este capítulo desarrolla la monografía completa (Bloques A – H) de **Haskell (GHC 9.x)**. Estudiar Haskell enseña en su forma más pura los conceptos que posteriormente han adoptado Rust (`Trait`, `Option`, `Result`), Scala, Swift y TypeScript: tipos algebraicos, clases de tipos, composición monádica y pruebas basadas en propiedades.*

Consejo

En Haskell de producción, utiliza siempre `newtype` en lugar de `data` cuando envuelvas un único campo con un solo constructor (`newtype CustomerId = CustomerId Int`): `newtype` garantiza distinción de tipos en compilación pero se borra por completo en ejecución con cero sobrecoste de puntero.

Definición

Forma Normal Débil a la Cabeza (Weak Head Normal Form, WHNF en Haskell): estado de reducción de una expresión en el que su capa más externa ya no es un redex (es un constructor de datos como `Just thunk`, un literal primitivo o una abstracción lambda `\x -> ...`), aunque sus subexpresiones internas permanezcan sin evaluar.

Autoficha

¿Por qué el sistema de tipos de Haskell impide ejecutar `putStrLn " enviando misil "` dentro de una transacción `atomically :: STM a -> IO a` de memoria transaccional por software?

4.1 Bloques A–D: Filosofía pura de Haskell, GHC/Cabal/Stack, currificación, evaluación perezosa, ADTs y Typeclasses

En **Haskell**, toda función es matemática y **referencialmente transparente**: para una misma entrada, produce siempre la misma salida sin mutar variables globales ni realizar entrada/salida oculta. Todas las funciones de varios argumentos están currificadas por defecto (`add :: Int -> Int -> Int` es `Int -> (Int -> Int)`), permitiendo la aplicación parcial directa (`map (+10) [1, 2, 3]`). La estrategia de evaluación por defecto es no estricta (*lazy evaluation* / *call-by-need*): las expresiones no se evalúan al pasarse como argumento, sino que se empaquetan en una suspensión en el montículo denominada **thunk** que se reduce a *Weak Head Normal Form* (WHNF) solo cuando la coincidencia de patrones exige conocer su constructor exterior, compartiendo el resultado para no recalcularlo. Cuando se acumulan sumas en un pliegue izquierdo, usar `foldl` perezoso construye una cadena de N *thunks* `(((0 + x1) + x2) ...)` que desborda la memoria (*space leak*); la solución profesional consiste en usar siempre la variante estricta `Data.List.foldl'`.

4.2 Bloques E–H: Functor, Applicative, Monad (IO, Reader, State, Either), concurrencia STM, QuickCheck y proyectos

Haskell parametriza constructores de tipos de *kind* `* -> *` mediante la jerarquía algebraica `Functor` (`fmap :: (a -> b) -> f a -> f b`), `Applicative` (`pure :: a -> f a`, `(<*>) :: f (a -> b) -> f a -> f b`) y **`Monad`** (`(>>=) :: m a -> (a -> m b) -> m b`). Toda instancia de `Monad` debe satisfacer las tres leyes monádicas: identidad por la izquierda (`return a >>= k == k a`), identidad por la derecha

(`m >>= return == m`) y asociatividad (`(m >>= k) >>= h == m >>= (\x -> k x >>= h)`). La notación `do` es azúcar sintáctico directo sobre `(>>=)`:

```
module SafeEval (Expr(..), eval) where

data Expr = Lit Int | Add Expr Expr | Div Expr Expr
  deriving (Show, Eq)

eval :: Expr -> Either String Int
eval (Lit n)     = Right n
eval (Add e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  Right (v1 + v2)
eval (Div e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  if v2 == 0 then Left "division by zero" else Right (div v1 v2)
```

Gracias a que las lecturas y escrituras de estado están controladas por el sistema de tipos, Haskell ofrece **Software Transactional Memory (STM)**: transacciones en memoria (`TVar`, `atomically`, `retry`, `orElse`) que se componen sin posibilidad de interbloqueos (*deadlocks*), ya que el tipo `STM a` prohíbe estáticamente ejecutar efectos `IO` irreversibles dentro de un bloque que podría reintentarse.

Problema resuelto

Enunciado En GHC de 64 bits, cada celda `(:)` de una lista enlazada `[Int]` en el montículo ocupa 3 palabras de 8 B (24 B por nodo). Una tubería `sum (map (*2) (map (+1) [1..N]))` para $N = 100000$ elementos, si se evaluara de forma estricta sin fusión, alojaría 3 listas de 100000 celdas ($3 \times 100000 \times 24$ B). Gracias a la fusión `foldr/build` de GHC con `-O2`, las tres listas desaparecen y el bucle se ejecuta en registros alojando 0 B de celdas. Calcula cuántos kilobytes (dividiendo entre 1000) ahorra la desforestación.

Datos

- $k_{listas} = 3$ — Listas intermedias eliminadas
- $N = 100000$ — Elementos de la secuencia
- $S_{cons} = 24\,\mathrm{B}$ — Tamaño de una celda cons en GHC 64 bits

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de montículo mediante desforestación foldr/build en GHC

$$\Delta M_{\mathrm{kB}} = \frac{3 \cdot 100000 \cdot 24}{1000}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos 3 * 100000 * 24 B y dividimos entre 1000 B/kB

$$\Delta M_{\mathrm{kB}} = \frac{3 \cdot 100000 \cdot 24}{1000}$$

$$\frac{((3 \cdot 100000) \cdot 24)}{1000}$$

Resultado

$$\mathit{Resultado} = 7200$$

Interpretación La fusión de flujos de GHC evita alojar 7200 kB (7.2 MB) de celdas intermedias.

Comprobación de los pasos

- Paso 1: 7200 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0021**] **(Básico, Numérico)** En el evaluador monádico `eval (Div (Add (Lit 20) (Lit 22)) (Lit 6))`, ¿qué valor entero `Right v` devuelve la expresión $((20 + 22)/6)$?
2 [**PRB-0022**] **(Básico, Numérico)** `QuickCheck` ejecuta por defecto 100 casos generados aleatoriamente por cada propiedad `prop_*`. Si un módulo define 18 propiedades, ¿cuántos casos de prueba verifica `quickCheck` en total?
3 [**PRB-0023**] **(Intermedio, Desarrollo por pasos)** En la expresión aplicativa `[(+10), (*2), (+100)] <*> [1, 2, 3, 4]`, la instancia `Applicative` de listas aplica cada una de las 3 funciones a cada uno de los 4 valores. ¿Cuántos elementos contiene la lista resultante?
4 [**PRB-0024**] **(Intermedio, Diseño)** Un `newtype` en Haskell envuelve un `Int64` (8 B). ¿Cuántos bytes de sobrecoste adicional añade el constructor del `newtype` en tiempo de ejecución tras el borrado de GHC?

Capítulo 5

Monografías de OCaml 5.x y F# (.NET) (Bloques A–H): Módulos de Orden Superior (Functors), Efectos Algebraicos y Expresiones de Computación

*Este capítulo presenta las monografías completas (Bloques $A - H$) de los dos principales exponentes industriales de la familia **ML** (*Meta Language*): **OCaml 5.x** (el lenguaje de elección para escribir compiladores, demostradores de teoremas como Coq/Rocq y sistemas de negociación algorítmica de baja latencia como en Jane Street) y **F#** (el lenguaje funcional primero del ecosistema .NET).*

Consejo

En OCaml, define siempre un fichero de interfaz explícito `.mli` para cada módulo `.ml`: mantener abstractos los tipos internos en el `.mli` garantiza el encapsulamiento de invariantes y acelera la compilación incremental con `dune`.

Definición

Entero etiquetado de 63 bits en OCaml (Tagged Integer): representación en máquina de 64 bits en la que un entero matemático n se codifica en registro como la palabra $(2n + 1)$ (desplazado 1 bit a la izquierda con el bit menos significativo fijado a `1`), permitiendo al recolector de basura distinguirlo instantáneamente de cualquier puntero alineado (cuyo bit 0 es `0`) sin alojar en el montículo.

Autoficha

¿Por qué la codificación de un entero n como $2n+1$ en OCaml permite que la suma de dos enteros etiquetados se implemente en ensamblador como `(a + b) - 1` en un solo ciclo de reloj sin desencajonar punteros?

5.1 Monografía de OCaml 5.x (Bloques A–H): Inferencia ML, Functors de módulos, representación en memoria y efectos algebraicos

OCaml (desarrollado en INRIA por Xavier Leroy y su equipo) combina evaluación estricta predecible, un compilador nativo rapidísimo (`ocamlopt`) con un recolector generacional incremental de latencias sub-milisegundo y el sistema de módulos más potente de la industria: en OCaml, los módulos (`struct ... end`) tienen tipos llamados firmas (`sig ... end`), y un **Functor** es una función a nivel de módulos que recibe un módulo como argumento y devuelve un nuevo módulo especializado (por ejemplo, `Map.Make(String)` construye un árbol AVL tipado para claves `string`). En cuanto a su representación de bajo nivel en 64 bits, OCaml reserva el bit menos significativo de cada palabra como etiqueta del recolector de basura (`1` para enteros inmediatos sin puntero `int` de **63 bits**, y `0` para punteros alineados al montículo). Además, **OCaml 5** incorpora paralelismo multinúcleo real (`Domain.spawn`) y **manejadores de efectos algebraicos** (*Effect Handlers*), que permiten definir planificadores asíncronos como bibliotecas puras sin colorear las funciones con `async`.

5.2 Monografía de F# sobre .NET (Bloques A–H): Tuberías, unidades de medida ([<Measure>]), Computation Expressions y proyectos

Por su parte, **F#** (diseñado por Don Syme en Microsoft Research) traslada la elegancia de OCaml al CLR de .NET. Entre sus innovaciones más notables destacan las **unidades de medida verificadas en compilación** (`[<Measure>] type m`, `[<Measure>] type s`, `[<Measure>] type N = kg m / s^2`), que previenen errores dimensionales (como el accidente de la sonda Mars Climate Orbiter) y se borran por completo al emitir el bytecode CIL con coste cero en tiempo de ejecución:

```
[<Measure>] type m
[<Measure>] type s
[<Measure>] type kg

let kineticEnergy (mass: float<kg>) (velocity: float<m/s>) : float<kg * m^2 / s^2> =
    0.5 * mass * velocity * velocity
```

Asimismo, F# generaliza las mónadas, los generadores y las consultas mediante una construcción sintáctica unificada: las **expresiones de computación** (*Computation Expressions*: `seq { yield ... }`, `async { let! x = ... }`, `task { ... }`, `result { ... }`).

Problema resuelto

Enunciado En el compilador nativo `ocamlopt` de 64 bits, cualquier entero `int` n se almacena en un registro de CPU mediante la representación etiquetada $R(n) = 2n + 1$ (con el bit 0 puesto a 1 para indicar al recolector de basura que no es un puntero). Calcula el valor decimal exacto de la palabra en registro $R(n)$ para el entero de aplicación $n = 1500$.

Datos

- $n = 1500$ — Valor entero lógico en el programa OCaml

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Codificación de enteros inmediatos de 63 bits (2n + 1) en OCaml

$$R(1500) = 2 \cdot 1500 + 1$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Desplazamos 1 bit a la izquierda (multiplicar por 2) y activamos el bit de etiqueta (+1)

$$R(1500) = 2 \cdot 1500 + 1$$

$$((2 \cdot 1500) + 1)$$

Resultado

$$\mathit{Resultado} = 3001$$

Interpretación El registro de CPU almacena el valor impar 3001 (0xBB9), que el recolector identifica en O(1) como un valor inmediato no rastreable.

Comprobación de los pasos

- Paso 1: 3001 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0027**] **(Básico, Numérico)** En OCaml, si un registro contiene la palabra etiquetada $R(n) = 85$, ¿cuál es el entero lógico $n = (85 - 1)/2$?

2 [**PRB-0028**] **(Básico, Numérico)** En F#, la función `kineticEnergy` calcula $0{,}5 \cdot m \cdot v^2$. Para una masa $m = 4$ kg y velocidad $v = 15$ m/s, ¿cuántos julios ($\mathrm{kg \cdot m^2/s^2}$) devuelve?

3 [**PRB-0029**] **(Intermedio, Desarrollo por pasos)** ¿Cuántos bytes adicionales ocupa en el ensamblado `.dll` de .NET la anotación de unidades `[<Measure>] type kg` tras la compilación de F#?

4 [**PRB-0030**] **(Intermedio, Diseño)** En OCaml de 64 bits, cada bloque en el montículo tiene 1 palabra de cabecera (8 B) que codifica tamaño, color GC y etiqueta de constructor, seguida de k campos de 8 B. ¿Cuántos bytes ocupa un nodo de árbol binario con 3 campos (`valor`, `izq`, `der`)?

Capítulo 6

Monografías de Elixir, Erlang/OTP (BEAM), Scala 3, Clojure y Lisp/Scheme (Bloques A–H)

*Este capítulo desarrolla las monografías completas (Bloques A–H) de cinco lenguajes fundamentales de la computación concurrente y funcional: **Erlang/OTP** y **Elixir** (sobre la máquina virtual BEAM), **Scala 3**, **Clojure** y **Lisp/Scheme**.*

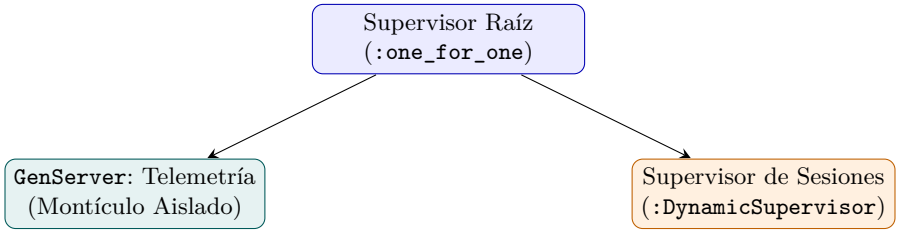


Figura 2. Árbol de supervisión OTP en Erlang/Elixir: los supervisores monitorizan procesos trabajadores (GenServer) aislados en memoria y aplican estrategias deterministas de reinicio (one_for_one, rest_for_one, one_for_all).

Consejo

En Erlang y Elixir, evita acumular millones de mensajes sin procesar en el buzón (*mailbox*) de un único `GenServer` usando `cast` asíncrono sin control: utiliza `GenServer.call` (que aplica contrapresión síncrona al llamador) o procesadores con demanda controlada como `GenStage` y `Broadway`.

Definición

Homoiconicidad (Code-as-Data en la familia Lisp): propiedad de un lenguaje cuya representación sintáctica primaria (S-expressions en Common Lisp/Scheme y EDN en Clojure) coincide directamente con las estructuras de datos primitivas manipulables por el propio lenguaje, haciendo trivial la metaprogramación por macros.

Autoficha

¿Por qué en un vector persistente inmutable de Clojure con factor de ramificación $B = 32$, acceder o actualizar un elemento entre $32^4 = 1048576$ elementos requiere recorrer únicamente 4 niveles de punteros?

6.1 Monografías de Erlang/OTP y Elixir sobre la BEAM (Bloques A–H): Actores aislados, GenServer y Supervisores

Creado por Joe Armstrong, Robert Virding y Mike Williams en Ericsson para conmutadores telefónicos con una disponibilidad del 99,9999999 % (*nueve nueves*), **Erlang** y su compañero moderno **Elixir** (creado por José Valim) se ejecutan sobre la máquina virtual **BEAM**. En la BEAM, cada actor (proceso Erlang/Elixir, que ocupa apenas ≈ 2,6 KiB iniciales) posee su **propio montículo privado inmutable** y su propio recolector de basura independiente: no existe memoria mutable compartida ni pausas globales *Stop-The-World* que detengan a todo el nodo. La tolerancia a fallos se estructura mediante **OTP** (*Open Telecom Platform*) y la filosofía *«Let it crash»*: como ilustra la Figura \ref{fig:fig-otp-supervision-tree}, cuando un proceso trabajador (`GenServer`) encuentra un estado corrup-

to inesperado, muere limpiamente liberando su montículo instantáneamente y su `Supervisor` padre lo reinicia a un estado inicial conocido válido:

```
defmodule TelemetryCounter do
  use GenServer

  def start_link(initial \\ 0), do: GenServer.start_link(__MODULE__, initial, name: __MODULE__
    )
  def increment(delta),          do: GenServer.call(__MODULE__, {:inc, delta})

  @impl true
  def init(initial), do: {:ok, initial}

  @impl true
  def handle_call({:inc, delta}, _from, state) do
    new_state = state + delta
    {:reply, new_state, new_state}
  end
end
```

6.2 Monografías de Scala 3, Clojure y Common Lisp / Scheme (Bloques A–H): Tipos dependientes, HAMT y macros

Scala 3 (Martin Odersky, EPFL) unifica la programación orientada a objetos y funcional sobre el cálculo **DOT** (*Dependent Object Types*), reemplazando los antiguos `implicit` sobrecargados de Scala 2 por las abstracciones contextuales limpias `given` y `using`, tipos unión (`A | B`) e intersección (`A & B`) y enumeraciones algebraicas (`enum`). Por su parte, **Lisp** (John McCarthy, 1958; estandarizado en **Common Lisp** y **Scheme** R7RS) y **Clojure** (Rich Hickey, sobre la JVM) se distinguen por la **homoiconicidad** (*el código es datos*: el programa se escribe directamente como listas de símbolos o vectores que son el propio AST del lenguaje), lo que permite escribir macros sintácticas que transforman el lenguaje desde dentro. Clojure aporta además vectores y mapas persistentes inmutables implementados como árboles trie de factor de ramificación 32 (**HAMT**, *Hash Array Mapped Trie* con compartición estructural $O(\log_{32} N)$), donde producir una versión modificada de un vector de un millón de elementos copia a lo sumo $\lceil \log_{32}(10^6) \rceil = 4$ nodos internos.

Problema resuelto

Enunciado En el vector persistente inmutable de Clojure y Scala (`Vector`), cada nodo interno contiene 32 punteros. Para un vector con $N = 1048576$ elementos (32^4, profundidad $d = 4$ niveles), crear un nuevo vector con un elemento actualizado mediante **compartición estructural** (*path copying*) clona únicamente los $d = 4$ nodos de 32 referencias a lo largo del camino modificado, reutilizando todos los demás nodos. Calcula cuántas referencias de puntero se copian en esa actualización (4×32).

Datos

- $d = 4$ niveles — Profundidad del trie para $32^4 = 1048576$ elementos
- $B = 32$ punteros — Factor de ramificación por nodo

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Coste de actualización por Path Copying en vectores persistentes de ramificación 32

$$P_{\text{copiados}} = 4 \cdot 32$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 4 niveles modificados por las 32 ranuras de cada nodo

$$P_{\text{copiados}} = 4 \cdot 32$$

$$(4 \cdot 32)$$

Resultado

$$Resultado = 128$$

Interpretación Actualizar inmutablemente un vector de más de un millón de elementos copia únicamente 128 punteros, compartiendo el 99.99 % de la estructura.
Comprobación de los pasos

- Paso 1: 128 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0033] (Básico, Numérico) Un nodo de la máquina virtual BEAM ejecuta 25 000 procesos Elixir aislados, cada uno con un montículo inicial de 3 KiB. ¿Cuántos KiB suman en total esos 25 000 procesos?
2 [PRB-0034] (Básico, Numérico) En el planificador preventivo de la BEAM, cada proceso dispone de un presupuesto de `2000` reducciones (llamadas a función) antes de ceder la CPU a otro proceso. Si un proceso ejecuta 14 000 llamadas a función, ¿cuántas rodajas de planificación (*context switches*) consume?
3 [PRB-0035] (Intermedio, Desarrollo por pasos) En un árbol persistente de Clojure con factor de ramificación 32, ¿cuántos elementos caben en $d = 3$ niveles (32^3)?
4 [PRB-0036] (Intermedio, Diseño) Un supervisor OTP configurado con estrategia `:one_for_all` supervisa 6 procesos trabajadores y uno de ellos falla. ¿Cuántos procesos trabajadores reinicia el supervisor?

Parte III

Monografías de Lenguajes Científicos, Embebidos y Lógicos (Julia, R, Lua, Perl, Objective-C, Prolog, GDScript)

Capítulo 7

Monografías de Julia y R (Bloques A–H): Despacho Múltiple, Especialización JIT LLVM, Vectorización y Ciencia de Datos

*Este capítulo desarrolla las monografías completas (Bloques $A - H$) de los dos lenguajes especializados en cálculo numérico, simulación física y estadística avanzada: **Julia** y **R**.*

Consejo

En estructuras `struct` de Julia que van a participar en bucles numéricos intensivos, nunca declares campos con tipos abstractos (`x::Real` o `v::Vector{Number}`): parametriza siempre la estructura (`struct Particle{T<:Real} x::T end`) para que el compilador conozca el tamaño exacto del campo y lo aloje en línea sin punteros.

Definición

Estabilidad de Tipos (Type Stability en Julia): propiedad de un método según la cual el compilador puede inferir un único tipo concreto (no abstracto ni unión abierta `Any`) para su valor de retorno y todas sus variables internas a partir únicamente de los tipos de los argumentos de entrada.

Autoficha

¿Por qué en Julia la función `f(x) = x > 0 ? x : 0.0` es inestable en tipos cuando se llama con un `Int64` (porque una rama devuelve `Int64` y la otra `Float64`), y cómo la corrige `zero(x)`?

7.1 Monografía de Julia (Bloques A–H): Despacho múltiple, estabilidad de tipos, broadcasting fusionado y HPC

Creado en el MIT por Jeff Bezanson, Stefan Karpinski, Viral B. Shah y Alan Edelman, **Julia** fue diseñado para eliminar el «problema de los dos lenguajes» (prototipar en un lenguaje dinámico lento y reescribir en C/Fortran para producción). El núcleo semántico de Julia es el **despacho múltiple** (*multiple dispatch*): las funciones genéricas seleccionan el método más específico considerando simultáneamente el tipo concreto de **todos** sus argumentos, y cuando se invoca un método con una tupla de tipos concretos, el compilador JIT genera código máquina LLVM especializado para esa firma exacta. La regla de oro del rendimiento en Julia es la **estabilidad de tipos** (*type stability*, verificable con `@code_warntype`): el tipo del valor de retorno de una función debe depender exclusivamente de los tipos de sus argumentos y nunca de sus valores en tiempo de ejecución. Además, la sintaxis de difusión con punto `y .= a .* x .+ b` fusiona a nivel sintáctico múltiples operaciones elementales en un único bucle SIMD en el sitio sin alojar ningún array temporal:

```
abstract type DissipationLaw end
struct LinearDrag <: DissipationLaw
    coeff::Float64
end

# Despacho múltiple especializado y bucle fusionado sin alojamientos (0 bytes):
function apply_damping!(v::AbstractVector{Float64}, law::LinearDrag, dt::Float64)
    @. v = v * (1.0 - law.coeff * dt)
    return v
end
```

7.2 Monografía de R (Bloques A−H): Vectores, data frames, fórmulas estadísticas, Tidyverse/data.table y Quarto

Por su parte, **R** (Ross Ihaka y Robert Gentleman, heredero del lenguaje S de John Chambers en Bell Labs) es el estándar en bioestadística, econometría y ciencia de datos reproducible. En R no existen escalares aislados: el número `42` es un vector numérico de longitud 1, todas las operaciones aritméticas están vectorizadas en C/Fortran bajo el capó, los valores faltantes tipados `NA` (*Not Available*) son ciudadanos de primera clase y los objetos siguen semántica funcional **Copy-on-Modify**. Asimismo, R posee evaluación perezosa de argumentos mediante promesas (*Non-Standard Evaluation*, NSE) y un operador de fórmulas simbólicas (`y ~ x1 + x2 + x1:x2`) que construye matrices de diseño para regresión lineal (`lm`), modelos lineales generalizados (`glm`) y efectos mixtos.

Problema resuelto

Enunciado Una expresión vectorial `w = a .* x .+ b .* y .- c` contiene $k = 4$ operaciones binarias sobre vectores de $N = 1000000$ de elementos `Float64` (8 B cada uno). En un lenguaje sin fusión de expresiones, evaluar los 4 operadores materializaría 4 vectores temporales en memoria ($4 \times 10^6 \times 8$ B). En Julia, al escribir `@. w = a * x + b * y - c` sobre un vector preasignado `w`, la fusión sintáctica aloja 0 B. Calcula cuántos megabytes exactos (dividiendo entre 10^6) ahorra la fusión de Julia.
Datos

- $k = 4$ operaciones — Vectores temporales evitados
- $N = 1000000$ — Longitud de los vectores Float64
- $S_{f64} = 8$ B — Tamaño de cada Float64

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Ahorro de asignaciones mediante Loop Fusion de broadcasting en Julia

$$\Delta M_{\text{MB}} = \frac{4 \cdot 10^6 \cdot 8}{10^6}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos $4 * 10^6 * 8B$ y dividimos entre $10^6 B/MB$

$$\Delta M_{\text{MB}} = \frac{4 \cdot 10^6 \cdot 8}{10^6}$$

$$\frac{((4 \cdot 1000000) \cdot 8)}{1000000}$$

Resultado

$$Resultado = 32$$

Interpretación La fusión sintáctica de Julia ahorra 32 MB de asignaciones temporales por evaluación y maximiza la localidad en caché L1/L2.
Comprobación de los pasos

- Paso 1: 32 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0039**] **(Básico, Numérico)** En Julia, si una función genérica tiene 4 métodos especializados para el primer argumento y 4 para el segundo (todas las combinaciones de pares), ¿cuántos métodos especializados forman la tabla de despacho (4×4)?

2 [**PRB-0040**] **(Básico, Numérico)** En R, un `data.frame` tiene 12 columnas numéricas (`double` de 8 B) y 50 000 filas. ¿Cuántos bytes suman los datos de las 12 columnas $(12 \times 50000 \times 8)$?

3 [**PRB-0041**] **(Intermedio, Desarrollo por pasos)** En un modelo lineal en R con intercepto y $p = 5$ predictores continuos más todas las interacciones de pares entre los 5 predictores $(5 \times 4/2 = 10)$, ¿cuántos coeficientes β estima en total `lm()` ($1 + 5 + 10$)?

4 [**PRB-0042**] **(Intermedio, Diseño)** Una validación cruzada de $K = 10$ pliegues (*10-fold CV*) repetida 5 veces evalúa una rejilla de 8 hiperparámetros. ¿Cuántos ajustes del modelo ejecuta en total $(10 \times 5 \times 8)$?

Capítulo 8

Monografías de Lua 5.4+, Perl 5, Objective-C, Prolog y GDScript (Bloques A–H): Scripting Embebido, Unificación Lógica y Motores

*Este capítulo cierra las monografías de lenguajes de la Parte V completando los Bloques $A - H$ para **Lua**, **Perl**, **Objective-C**, **Prolog** y **GDScript**.*

Consejo

En GDScript (Godot 4), añade anotaciones de tipos estáticos explícitas (`var vel: Vector2`, `func calc(x: float) -> float:`) en todos los scripts de física e inteligencia artificial: permite a la máquina virtual de Godot emitir opcodes tipados directos evitando la resolución dinámica de variantes en cada fotograma.

Definición

Unificador General Máximo (Most General Unifier, MGU en Prolog): sustitución de variables θ tal que, dados dos términos lógicos s y t, los hace sintácticamente idénticos ($s\theta = t\theta$) y cualquier otro unificador σ de s y t es una instancia de θ ($\sigma = \theta\gamma$).

Autoficha

¿Qué diferencia fundamental existe entre la asignación destructiva `x = x + 1` de un lenguaje imperativo y la unificación `X = Y + 1` frente a la evaluación aritmética `X is Y + 1` en Prolog?

8.1 Monografías de Lua 5.4+, Perl 5 y Objective-C (Bloques A–H): Tablas/Metatablas, Regex y objc_msgSend

Diseñado en la PUC-Rio (Roberto Ierusalimschy, Waldemar Celes y Luiz Henrique de Figueiredo), **Lua** es el lenguaje de scripting embebido por excelencia en motores de videojuegos, Redis, Nginx/OpenResty y Neovim: todo su intérprete ANSI C ocupa menos de 300 KiB y se ejecuta sobre una máquina virtual basada en registros. La única estructura de datos compuesta de Lua es la **tabla**, implementada internamente como una estructura híbrida que combina una parte de array denso (indexada desde `1`) y una parte de tabla hash; asociando a una tabla una **metatabla** con metamétodos como `__index` y `__call`, se implementan sistemas de objetos, proxies y sobrecarga de operadores:

```
local Vector2D = {}
Vector2D.__index = Vector2D

function Vector2D.new(x, y)
    return setmetatable({ x = x or 0, y = y or 0 }, Vector2D)
end

function Vector2D:magnitude_sq()
    return self.x * self.x + self.y * self.y
end
```

En **Perl 5** (Larry Wall), todo módulo moderno comienza con `use v5.36; use strict; use warnings;` y distingue por sigilo e invocación entre contexto escalar (`$x`), lista (`@arr`) y tabla asociativa

(`%hash`), habiendo definido el estándar moderno de expresiones regulares (PCRE). En **Objective-C** (Brad Cox), las llamadas `[receiver message:arg]` no se resuelven por desplazamiento de tabla virtual fija como en C++, sino mediante el despachador dinámico en tiempo de ejecución `objc_msgSend(receiver, @selector(message:), arg)`, lo que permite enviar mensajes a `nil` de forma segura (no-op que devuelve cero) e introspeccionar selectores dinámicamente.

8.2 Monografías de Prolog y GDScript (Bloques A–H): Unificación lógica (SLD) y programación de motores de videojuegos

En el paradigma lógico declarativo, un programa en **Prolog** (Alain Colmerauer y Philippe Roussel, 1972; ISO Prolog / SWI-Prolog) consta de una base de conocimientos de **cláusulas de Horn**: hechos (`padre(juan, maria).`) y reglas (`ancestro(X, Y) :- padre(X, Z), ancestro(Z, Y).`). Ante una consulta, el motor ejecuta **resolución SLD** combinando el algoritmo de **unificación sintáctica de Robinson** (que calcula el unificador general máximo, MGU, vinculando variables lógicas en ambos sentidos) con búsqueda en profundidad y retroceso (*backtracking*) automático, además de aritmética con restricciones sobre dominios finitos (`use_module(library(clpfd))`). Finalmente, **GDScript** (el lenguaje específico de dominio del motor libre **Godot Engine 4.x**) ofrece tipado gradual estático (`var speed: float = 250.0`), referencias directas a nodos del árbol de escenas y comunicación desacoplada mediante el patrón Observador con señales tipadas (`signal health_changed(new_hp: int)`), separando estrictamente el bucle visual (`_process(delta)`) del paso fijo del motor de física (`_physics_process(delta)`).

Problema resuelto

Enunciado En la implementación C de las tablas híbridas de Lua 5.4, cuando una tabla almacena claves enteras consecutivas `1..N`, se alojan en la parte array (`TValue` de 16 B por elemento: 8 B de valor más 1 B de etiqueta alineado), mientras que en la parte hash cada nodo (`Node`) ocupa 32 B (clave + valor + enlace `next`). Calcula cuántos bytes ahorra una tabla Lua de $N = 4000$ elementos al indexarse secuencialmente desde `1` hasta `4000` (parte array de 16 B) frente a usar claves dispersas en la parte hash (32 B por nodo).

Datos

- $N = 4000$ — Elementos en la tabla Lua
- $S_{hash} = 32\,\mathrm{B}$ — Tamaño por nodo en la parte hash
- $S_{array} = 16\,\mathrm{B}$ — Tamaño por TValue en la parte array

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de memoria de la parte Array frente a la parte Hash en tablas híbridas de Lua 5.4

$$\Delta M = 4000 \cdot (32 - 16)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 4000 elementos por los 16 B ahorrados al residir en la parte array

$$\Delta M = 4000 \cdot (32 - 16)$$

$$(4000 \cdot (32 - 16))$$

Resultado

$$\mathit{Resultado} = 64000$$

Interpretación Usar índices enteros consecutivos desde 1 ahorra 64 000 bytes (la mitad exacta de memoria) y evita el cálculo de hashes.

Comprobación de los pasos

- Paso 1: 64000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0045**] **(Básico, Numérico)** En el ejemplo de Lua `Vector2D.new(6, 8):magnitude_sq()`, ¿qué valor devuelve $6^2 + 8^2$?

2 [**PRB-0046**] **(Básico, Numérico)** En Godot Engine (GDScript), el bucle `_physics_process(delta)` se ejecuta por defecto a una frecuencia fija de 60 tics por segundo. ¿Cuántos pasos de física ejecuta exactamente en 15 segundos (60×15)?

3 [**PRB-0047**] **(Intermedio, Desarrollo por pasos)** Un predicado no determinista en Prolog tiene 3 cláusulas que se ramifican en profundidad $d = 4$ niveles (3^4 hojas en el árbol de búsqueda SLD). Si el operador de corte `!` poda 2 de las 3 ramas en la raíz, ¿cuántas hojas se ahorra explorar ($3^4 - 3^3$)?

4 [**PRB-0048**] **(Intermedio, Diseño)** En la API de pila en C de Lua (`lua_State`), una función nativa empuja 5 argumentos a la pila, invoca `lua_pcall(L, 4, 2, 0)` (que consume la función más los 4 argumentos y empuja 2 resultados). Si antes de empezar había 3 elementos en la pila, ¿cuántos elementos hay en la pila al terminar ($3 + 2$)?

Apéndice A

Notación

Símbolo	Significado	Unidades
$\sigma_p(R), \pi_L(R), R \bowtie S$	Operadores de selección, proyección y reunión natural del álgebra relacional	relación
$m \gg= f : M\langle B \rangle$	Operador bind (»=) de una mónada en programación funcional	tipo
w OVER (P, O)	Marco de evaluación de una función de ventana en SQL estándar	ventana

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Monografía de SQL (Bloques A–H): Álgebra Relacional, DDL/DML, Lógica Trivaluada, CTEs Recursivas, Funciones de Ventana y Optimización

Problema resuelto [PRB-0001]
En el proyecto `projects/sql-transactional-ledger`, la cuenta de Caja (`account_id = 110`) registra tres apuntes sucesivos `(debit_cents, credit_cents)`: `(500000, 0)`, `(125000, 0)` y `(0, 40000)`. Calcula el saldo acumulado en céntimos (`running_balance_cents`) que devuelve la función de ventana `SUM(debit_cents - credit_cents) OVER (...)` en la tercera fila.
Respuesta

$$\textit{Resultado} = 585000$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
Ante las puntuaciones `[100, 90, 90, 80]`, ¿qué valor entero asigna `RANK()` al cuarto elemento (`80`) tras el empate en la posición 2?
Respuesta 4
Ejercicio 2 [PRB-0004]
Ante las mismas puntuaciones `[100, 90, 90, 80]`, ¿qué valor entero asigna `DENSE_RANK()` al cuarto elemento (`80`)?
Respuesta 3
Ejercicio 3 [PRB-0005]
Un producto cartesiano `CROSS JOIN` entre una tabla de 120 tiendas y una tabla de 365 días del año genera todas las combinaciones `(tienda, día)`. ¿Cuántas filas produce?
Respuesta 43800
Ejercicio 4 [PRB-0006]
Una consulta con paginación `LIMIT 50 OFFSET 400` devuelve la página 9 de resultados. ¿Cuál es el número de orden (base 1) de la primera fila devuelta en esa página ($400 + 1$)?
Respuesta 401

Cálculos reproducidos

$(((500000 - 0) + (125000 - 0)) + (0 - 40000)) = 585000$

C.2 Monografía de Bash y POSIX Shell (Bloques A–H): Tuberías, Redirecciones, Expansiones Seguras, Trap, Herramientas UNIX y Proyectos

Problema resuelto [PRB-0007]
Un contenedor en un pipeline de despliegue termina abruptamente con código de salida `$? = 137` tras intervenir el *OOM Killer* del kernel Linux (`SIGKILL = 9`), y otro proceso aborta por violación de segmento (`SIGSEGV = 11`, código $128 + 11 = 139$). Calcula la suma de los números de las dos señales POSIX subyacentes obtenidas restando 128 a cada código de salida $((137 - 128) + (139 - 128))$.
Respuesta

$$\textit{Resultado} = 20$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0009]
Un script en ejecución es interrumpido por el usuario pulsando `Ctrl+C` (`SIGINT = 2`). ¿Qué código de salida `$?` reporta el shell ($128 + 2$)?
Respuesta 130

Ejercicio 2 [PRB-0010]
Un proceso recibe la señal de terminación ordenada `SIGTERM = 15` sin capturarla. ¿Cuál es su código de salida ($128 + 15$)?
Respuesta 143

Ejercicio 3 [PRB-0011]
Un comando `xargs -0 -n 50 -P 4` procesa una lista de 600 ficheros pasándolos en lotes de 50 ficheros por invocación. ¿Cuántos procesos hijos lanza en total `xargs` ($600 / 50$)?
Respuesta 12

Ejercicio 4 [PRB-0012]
Una máscara `umask 027` se resta de los permisos base `666` (octal) para la creación de ficheros regulares ($666 - 27$ en dígito a dígito: $6-0=6$, $6-2=4$, $6-7 \to 0$). ¿Cuál es el permiso octal resultante expresado como entero de tres dígitos?
Respuesta 640

Cálculos reproducidos

$((137-128)+(139-128)) = 20$

C.3 Monografías de PHP 8.3+, Ruby 3.3+ y Dart 3.x (Bloques A−H): Motores Zend, YJIT y Compilación AOT/JIT Multiplataforma

Problema resuelto [PRB-0013]
Un servidor de producción con 8192 MiB de RAM reserva 2048 MiB para el sistema operativo y el búfer de OPcache/Redis, dejando $M_{\text{avail}} = 6144$ MiB para el pool de trabajadores PHP-FPM. Si cada trabajador consume en carga punta $m_{\text{worker}} = 48$ MiB de memoria privada, calcula el valor óptimo de `pm.max_children` ($6144 / 48$).
Respuesta

$$Resultado = 128$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0015]
En Flutter/Dart, para sostener una tasa de refresco de pantalla de 120 Hz, el presupuesto total por fotograma (construcción + diseño + pintado) es 1000/120 ms. ¿Cuántos milisegundos caben en 12 fotogramas consecutivos?
Respuesta 100

Ejercicio 2 [PRB-0016]
En Ruby, la expresión `(1..10).select(&:even?).map { |x| x * 3 }.sum` suma el triple de los pares del 2 al 10 ($(2+4+6+8+10)\times 3$). ¿Cuál es el resultado?
Respuesta 90

Ejercicio 3 [PRB-0017]
Si OPcache evita recompilar 85 ficheros PHP por petición HTTP (ahorrando 0,4 ms por fichero), ¿cuántos milisegundos de CPU ahorra en cada petición ($85 \times 0{,}4$)?
Respuesta 34

Ejercicio 4 [PRB-0018]
Un Isolate de fondo en Dart procesa un lote de 400 imágenes en grupos de 25 imágenes devolviendo el progreso al Isolate principal tras cada grupo. ¿Cuántos mensajes de progreso envía?
Respuesta 16

Cálculos reproducidos

$\frac{6144}{48} = 128$

C.4 Monografía de Haskell (Bloques A–H): Pureza, Evaluación Perezosa, Inferencia Hindley-Milner, Typeclasses y Mónadas

Problema resuelto [PRB-0019]
En GHC de 64 bits, cada celda `(:)` de una lista enlazada `[Int]` en el montículo ocupa 3 palabras de 8 B (24 B por nodo). Una tubería `sum (map (*2) (map (+1) [1..N]))` para $N = 100000$ elementos, si se evaluara de forma estricta sin fusión, alojaría 3 listas de 100000 celdas ($3 \times 100000 \times 24$ B). Gracias a la fusión `foldr/build` de GHC con `-O2`, las tres listas desaparecen y el bucle se ejecuta en registros alojando 0 B de celdas. Calcula cuántos kilobytes (dividiendo entre 1000) ahorra la desforestación.
Respuesta

$$Resultado = 7200$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0021]
En el evaluador monádico `eval (Div (Add (Lit 20) (Lit 22)) (Lit 6))`, ¿qué valor entero `Right v` devuelve la expresión $((20 + 22)/6)$?
Respuesta 7
Ejercicio 2 [PRB-0022]
`QuickCheck` ejecuta por defecto 100 casos generados aleatoriamente por cada propiedad `prop_*`. Si un módulo define 18 propiedades, ¿cuántos casos de prueba verifica `quickCheck` en total?
Respuesta 1800
Ejercicio 3 [PRB-0023]
En la expresión aplicativa `[(+10), (*2), (+100)] <*> [1, 2, 3, 4]`, la instancia `Applicative` de listas aplica cada una de las 3 funciones a cada uno de los 4 valores. ¿Cuántos elementos contiene la lista resultante?
Respuesta 12
Ejercicio 4 [PRB-0024]
Un `newtype` en Haskell envuelve un `Int64` (8 B). ¿Cuántos bytes de sobrecoste adicional añade el constructor del `newtype` en tiempo de ejecución tras el borrado de GHC?
Respuesta 0

Cálculos reproducidos

$\frac{((3 \cdot 100000) \cdot 24)}{1000} = 7200$

C.5 Monografías de OCaml 5.x y F# (.NET) (Bloques A–H): Módulos de Orden Superior (Functors), Efectos Algebraicos y Expresiones de Computación

Problema resuelto [PRB-0025]
En el compilador nativo `ocamlopt` de 64 bits, cualquier entero `int` n se almacena en un registro de CPU mediante la representación etiquetada $R(n) = 2n + 1$ (con el bit 0 puesto a 1 para indicar al recolector de basura que no es un puntero). Calcula el valor decimal exacto de la palabra en registro $R(n)$ para el entero de aplicación $n = 1500$.
Respuesta

$$Resultado = 3001$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0027]
En OCaml, si un registro contiene la palabra etiquetada $R(n) = 85$, ¿cuál es el entero lógico $n = (85-1)/2$?
Respuesta 42

Ejercicio 2 [PRB-0028]
En F#, la función `kineticEnergy` calcula $0{,}5 \cdot m \cdot v^2$. Para una masa $m = 4$ kg y velocidad $v = 15$ m/s, ¿cuántos julios ($\mathrm{kg \cdot m^2/s^2}$) devuelve?
Respuesta 450

Ejercicio 3 [PRB-0029]
¿Cuántos bytes adicionales ocupa en el ensamblado `.dll` de .NET la anotación de unidades `[<Measure>] type kg` tras la compilación de F#?
Respuesta 0

Ejercicio 4 [PRB-0030]
En OCaml de 64 bits, cada bloque en el montículo tiene 1 palabra de cabecera (8 B) que codifica tamaño, color GC y etiqueta de constructor, seguida de k campos de 8 B. ¿Cuántos bytes ocupa un nodo de árbol binario con 3 campos (`valor`, `izq`, `der`)?
Respuesta 32

Cálculos reproducidos

$((2 \cdot 1500) + 1) = 3001$

C.6 Monografías de Elixir, Erlang/OTP (BEAM), Scala 3, Clojure y Lisp/Scheme (Bloques A–H)

Problema resuelto [PRB-0031]
En el vector persistente inmutable de Clojure y Scala (`Vector`), cada nodo interno contiene 32 punteros. Para un vector con $N = 1048576$ elementos (32^4, profundidad $d = 4$ niveles), crear un nuevo vector con un elemento actualizado mediante **compartición estructural** (*path copying*) clona únicamente los $d = 4$ nodos de 32 referencias a lo largo del camino modificado, reutilizando todos los demás nodos. Calcula cuántas referencias de puntero se copian en esa actualización (4×32).
Respuesta

$$\mathit{Resultado} = 128$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0033]
Un nodo de la máquina virtual BEAM ejecuta 25 000 procesos Elixir aislados, cada uno con un montículo inicial de 3 KiB. ¿Cuántos KiB suman en total esos 25 000 procesos?
Respuesta 75000

Ejercicio 2 [PRB-0034]
En el planificador preventivo de la BEAM, cada proceso dispone de un presupuesto de `2000` reducciones (llamadas a función) antes de ceder la CPU a otro proceso. Si un proceso ejecuta 14 000 llamadas a función, ¿cuántas rodajas de planificación (*context switches*) consume?
Respuesta 7

Ejercicio 3 [PRB-0035]
En un árbol persistente de Clojure con factor de ramificación 32, ¿cuántos elementos caben en $d = 3$ niveles (32^3)?
Respuesta 32768

Ejercicio 4 [PRB-0036]
Un supervisor OTP configurado con estrategia `:one_for_all` supervisa 6 procesos trabajadores y uno de ellos falla. ¿Cuántos procesos trabajadores reinicia el supervisor?
Respuesta 6

Cálculos reproducidos

$(4 \cdot 32) = 128$

C.7 Monografías de Julia y R (Bloques A–H): Despacho Múltiple, Especialización JIT LLVM, Vectorización y Ciencia de Datos

Problema resuelto [PRB-0037]
Una expresión vectorial `w = a .* x .+ b .* y .- c` contiene $k = 4$ operaciones binarias sobre vectores de $N = 1000000$ de elementos `Float64` (8 B cada uno). En un lenguaje sin fusión de expresiones, evaluar los 4 operadores materializaría 4 vectores temporales en memoria ($4 \times 10^6 \times 8$ B). En Julia, al escribir `@. w = a * x + b * y - c` sobre un vector preasignado `w`, la fusión sintáctica aloja 0 B. Calcula cuántos megabytes exactos (dividiendo entre 10^6) ahorra la fusión de Julia.
Respuesta

$$Resultado = 32$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0039]
En Julia, si una función genérica tiene 4 métodos especializados para el primer argumento y 4 para el segundo (todas las combinaciones de pares), ¿cuántos métodos especializados forman la tabla de despacho (4×4)?
Respuesta 16
Ejercicio 2 [PRB-0040]
En R, un `data.frame` tiene 12 columnas numéricas (`double` de 8 B) y 50 000 filas. ¿Cuántos bytes suman los datos de las 12 columnas $(12 \times 50000 \times 8)$?
Respuesta 4800000
Ejercicio 3 [PRB-0041]
En un modelo lineal en R con intercepto y $p = 5$ predictores continuos más todas las interacciones de pares entre los 5 predictores $(5 \times 4/2 = 10)$, ¿cuántos coeficientes β estima en total `lm()` ($1 + 5 + 10$)?
Respuesta 16
Ejercicio 4 [PRB-0042]
Una validación cruzada de $K = 10$ pliegues (*10-fold CV*) repetida 5 veces evalúa una rejilla de 8 hiperparámetros. ¿Cuántos ajustes del modelo ejecuta en total $(10 \times 5 \times 8)$?
Respuesta 400

Cálculos reproducidos

$\frac{((4 \cdot 1000000) \cdot 8)}{1000000} = 32$

C.8 Monografías de Lua 5.4+, Perl 5, Objective-C, Prolog y GDScript (Bloques A–H): Scripting Embebido, Unificación Lógica y Motores

Problema resuelto [PRB-0043]
En la implementación C de las tablas híbridas de Lua 5.4, cuando una tabla almacena claves enteras consecutivas `1..N`, se alojan en la parte array (`TValue` de 16 B por elemento: 8 B de valor más 1 B de etiqueta alineado), mientras que en la parte hash cada nodo (`Node`) ocupa 32 B (clave + valor + enlace `next`). Calcula cuántos bytes ahorra una tabla Lua de $N = 4000$ elementos al indexarse secuencialmente desde `1` hasta `4000` (parte array de 16 B) frente a usar claves dispersas en la parte hash (32 B por nodo).
Respuesta

$$Resultado = 64000$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0045]
En el ejemplo de Lua `Vector2D.new(6, 8):magnitude_sq()`, ¿qué valor devuelve $6^2 + 8^2$?
Respuesta 100

Ejercicio 2 [PRB-0046]
En Godot Engine (GDScript), el bucle `_physics_process(delta)` se ejecuta por defecto a una frecuencia fija de 60 tics por segundo. ¿Cuántos pasos de física ejecuta exactamente en 15 segundos (60×15)?
Respuesta 900

Ejercicio 3 [PRB-0047]
Un predicado no determinista en Prolog tiene 3 cláusulas que se ramifican en profundidad $d = 4$ niveles (3^4 hojas en el árbol de búsqueda SLD). Si el operador de corte `!` poda 2 de las 3 ramas en la raíz, ¿cuántas hojas se ahorra explorar ($3^4 - 3^3$)?
Respuesta 54

Ejercicio 4 [PRB-0048]
En la API de pila en C de Lua (`lua_State`), una función nativa empuja 5 argumentos a la pila, invoca `lua_pcall(L, 4, 2, 0)` (que consume la función más los 4 argumentos y empuja 2 resultados). Si antes de empezar había 3 elementos en la pila, ¿cuántos elementos hay en la pila al terminar ($3 + 2$)?
Respuesta 5

Cálculos reproducidos

$(4000 \cdot (32 - 16)) = 64000$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 36 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.
¶1: dos testimonios.
¶2: dos testimonios.
¶3: dos testimonios.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios.
¶21: dos testimonios.
¶22: dos testimonios.
¶23: dos testimonios.
¶24: dos testimonios.
¶25: dos testimonios.
¶26: dos testimonios.
¶27: dos testimonios.
¶28: dos testimonios.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios. Ecuaciones: 1 respaldada por una fuente.
¶34: dos testimonios.
¶35: dos testimonios.
¶36: dos testimonios. Ecuaciones: 1 reproducida numéricamente, 1 valor o condición.
¶37: dos testimonios.
¶38: dos testimonios.
¶39: dos testimonios.
¶40: dos testimonios.
¶41: dos testimonios.
¶42: dos testimonios.
¶43: dos testimonios.
¶44: dos testimonios.
¶45: dos testimonios.
¶46: dos testimonios. Ecuaciones: 2 respaldada por una fuente.
¶47: dos testimonios.
¶48: dos testimonios.

Referencia Normativa src-sql-2023 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6.
Referencia Normativa src-codd-1970 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6.
Referencia Normativa src-posix-2017 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12.
Referencia Normativa src-bryant-ohallaron (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12.
Referencia Normativa src-ecmascript-2024 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
Referencia Normativa src-unicode-15 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
Referencia Normativa src-pierce-tapl (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
Referencia Normativa src-knuth-taocp (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Referencia Normativa src-csharp-spec (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
Referencia Normativa src-kleppmann-ddia (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
Referencia Normativa src-iso-c17 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: cap01-monografia-sql-ddl-dml-cte-ventanas-optimizacion scientific
REV-0002: cap02-monografia-bash-posix-shell-automatizacion-sistemas scientific
REV-0003: cap03-monografias-php-ruby-dart-web-movil scientific
REV-0004: cap04-monografia-haskell-funcional-puro-tipos-monadas scientific
REV-0005: cap05-monografias-ocaml-fsharp-ml-sistemas-tipos scientific
REV-0006: cap06-monografias-beam-elixir-erlang-scala3-clojure-lisp scientific
REV-0007: cap07-monografias-julia-r-computacion-cientifica-estadistica scientific
REV-0008: cap08-monografias-lua-perl-objc-prolog-gdscript scientific

Apéndice E

Plan de estudio

Monografías de Datos, Automatización de Sistemas y Desarrollo Web/Móvil

✓Monografía de SQL (Bloques A−H): Álgebra Relacional, DDL/DML, Lógica Trivaluada, CTEs Recursivas, Funciones de Ventana y Optimización

- Comprender el fundamento relacional de SQL (E. F. Codd, 1970; estándar ISO/IEC 9075:2023) y el orden lógico exacto de evaluación de una consulta (‘FROM -> WHERE -> GROUP BY -> HAVING -> WINDOW -> SELECT -> DISTINCT -> ORDER BY -> LIMIT‘).
- Dominar la lógica trivaluada de ‘NULL‘ (‘TRUE‘, ‘FALSE‘, ‘UNKNOWN‘), todos los tipos de ‘JOIN‘ (incluso ‘LATERAL‘), subconsultas correlacionadas, CTEs recursivas (‘WITH RECURSIVE‘) y funciones de ventana (‘ROW_NUMBER‘, ‘RANK‘, ‘LAG‘, ‘SUM() OVER (...)‘).
- Interpretar planes de ejecución (‘EXPLAIN ANALYZE‘), diseñar índices compuestos sargables y verificar el libro mayor financiero ‘projects/sql-transactional-ledger‘.

2 conceptos planificados.

✓Monografía de Bash y POSIX Shell (Bloques A−H): Tuberías, Redirecciones, Expansiones Seguras, Trap, Herramientas UNIX y Proyectos

- Distinguir con rigor entre el estándar portable POSIX ‘sh‘ (IEEE Std 1003.1) y las extensiones de GNU Bash (‘[[...]]‘, arrays indexados/asociativos, sustitución de procesos ‘<(...)‘).
- Dominar los descriptores de fichero (‘0‘ stdin, ‘1‘ stdout, ‘2‘ stderr), tuberías (‘|‘), códigos de salida (‘$¿), entrecomillado seguro y expansiones de parámetros (‘${var:-def}‘, ‘${var:?err}‘).
- Escribir scripts de producción robustos con modo estricto (‘set -euo pipefail‘), limpieza atómica mediante ‘trap ... EXIT‘, integración con ‘find‘/‘xargs -0‘/‘jq‘/‘awk‘ y verificación con ‘shellcheck‘.

2 conceptos planificados.

✓Monografías de PHP 8.3+, Ruby 3.3+ y Dart 3.x (Bloques A−H): Motores Zend, YJIT y Compilación AOT/JIT Multiplataforma

- Dominar PHP moderno (8.3+): tipado estricto (‘declare(strict_types=1)‘), tipos de unión/intersección, ‘readonly class‘, ‘enum‘ respaldados, Fibers, OPcache/JIT, Composer, estándares PSR y Laravel/Symfony.
- Dominar Ruby (3.3+): modelo de objetos puro, bloques/‘Proc‘/lambdas, mixins con ‘Module‘, metaprogramación, compilador YJIT, concurrencia con Fibers/Ractors y ecosistema Bundler/RSpec/Rails.
- Dominar Dart (3.x): *sound null-safety*, modificadores de clase (‘sealed‘, ‘final‘, ‘interface‘), coincidencia de patrones, concurrencia por memoria aislada (*Isolates*) y compilación JIT/AOT para Flutter.

2 conceptos planificados.

Monografías de Lenguajes Funcionales Estáticos y Concurrentes (Haskell, OCaml, F#, Scala, BEAM, Lisp)

✓Monografía de Haskell (Bloques A−H): Pureza, Evaluación Perezosa, Inferencia Hindley-Milner, Typeclasses y Mónadas

- Comprender la transparencia referencial, la evaluación no estricta (perezosa por necesidad, *call-by-need* con *thunks*), la currificación universal y el sistema de tipos Hindley-Milner de GHC.
- Diseñar tipos de datos algebraicos (‘data‘, ‘newtype‘), sinónimos de tipos, polimorfismo de clases de tipos (‘Eq‘, ‘Ord‘, ‘Monoid‘) y tipos de orden superior (*Higher-Kinded Types*, ‘* -> *‘).
- Dominar la jerarquía ‘Functor‘ (‘fmap‘), ‘Applicative‘ (‘<*>‘) y ‘Monad‘ (‘»=‘, notación ‘do‘), el aislamiento puro de efectos con ‘IO‘, ‘STM‘ (*Software Transactional Memory*) y pruebas de propiedades con ‘QuickCheck‘.

2 conceptos planificados.

✓Monografías de OCaml 5.x y F# (.NET) (Bloques A−H): Módulos de Orden Superior (Functors), Efectos Algebraicos y Expresiones de Computación

- Dominar la familia ML (Robin Milner): evaluación estricta por valor con inferencia Hindley-Milner completa, tipos algebraicos, coincidencia de patrones y mutabilidad explícita (‘ref‘, ‘mutable‘).
- En OCaml 5.x: programar módulos parametrizados de orden superior (*Functors*), variantes polimórficas, paralelismo nativo con *Domains* y concurrencia mediante manejadores de efectos algebraicos (*Effect Handlers*).
- En F#: dominar el operador tubería (‘|>‘), unidades de medida estáticas (‘[<Measure>]‘), proveedores de tipos y expresiones de computación (‘async { ... }‘, ‘seq { ... }‘).

2 conceptos planificados.

✓Monografías de Elixir, Erlang/OTP (BEAM), Scala 3, Clojure y Lisp/Scheme (Bloques A−H)

- Comprender la máquina virtual BEAM (Erlang/Elixir): aislamiento total de memoria por proceso, recolección de basura por proceso, paso de mensajes asíncrono y árboles de supervisión OTP (*let it crash*).
- Dominar Elixir (tuberías ‘|>‘, protocolos, macros higiénicas, ‘GenServer‘, ‘Supervisor‘, ‘Phoenix LiveView‘) y Erlang/OTP (coincidencia de patrones binarios, actualización de código en caliente).
- Dominar Scala 3 (cálculo DOT, ‘enum‘, ‘given‘/‘using‘, tipos unión/intersección, ZIO/Cats Effect), Clojure (estructuras persistentes HAMT, átomos/STM, REPL) y Lisp/Scheme (homoiconicidad, S-expressions y macros ‘defmacro‘/‘syntax-rules‘).

2 conceptos planificados.

Monografías de Lenguajes Científicos, Embebidos y Lógicos (Julia, R, Lua, Perl, Objective-C, Prolog, GDScript)

✓Monografías de Julia y R (Bloques A−H): Despacho Múltiple, Especialización JIT LLVM, Vectorización y Ciencia de Datos

- Comprender cómo Julia resuelve el «problema de los dos lenguajes» mediante despacho múltiple (*multiple dispatch*), inferencia de estabilidad de tipos y compilación JIT sobre LLVM.
- Dominar en Julia los tipos paramétricos abstractos y concretos, la difusión vectorizada fusionada con punto (‘f.(x) .+ g.(y)‘), macros metaprogramáticas y álgebra lineal/ecuaciones diferenciales (‘DifferentialEquations.jl‘).

- Dominar R (4.x): vectores atómicos con semántica Copy-on-Modify, factores, matrices, ‘data.frame‘ / ‘tibble‘ / ‘data.table‘, programación funcional (‘purrr‘), modelado estadístico (‘y ~ x1 + x2‘) y visualización con la Gramática de Gráficos (‘ggplot2‘).

2 conceptos planificados.

✓Monografías de Lua 5.4+, Perl 5, Objective-C, Prolog y GDScript (Bloques A−H): Scripting Embebido, Unificación Lógica y Motores

- Dominar Lua (5.4+ / LuaJIT): máquina virtual basada en registros, tablas híbridas (array + hash), metatablas (‘__index‘, ‘__newindex‘, ‘__add‘), corrutinas asimétricas y la API de pila en C (‘lua_State*‘).
- Comprender Perl 5 (contexto escalar vs lista, expresiones regulares PCRE, CPAN, ‘use strict; use warnings;‘) y Objective-C (paso de mensajes dinámico Smalltalk ‘objc_msgSend‘, categorías y puente con Swift/C).
- Dominar Prolog (cláusulas de Horn, resolución SLD, unificación sintáctica de Robinson, retroceso y corte ‘¡) y GDScript (árbol de escenas de Godot, señales desacopladas y procesado por fotograma ‘_process‘/‘_physics_process‘).

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Referencia Normativa src-sql-2023. https://doc.rust-lang.org/reference/src-sql-2023.html

[27] Referencia Normativa src-codd-1970. https://doc.rust-lang.org/reference/src-codd-1970.html

[28] Referencia Normativa src-posix-2017. https://doc.rust-lang.org/reference/src-posix-2017.html

[29] Referencia Normativa src-bryant-ohallaron. https://doc.rust-lang.org/reference/src-bryant-ohallaron.html

[30] Referencia Normativa src-ecmascript-2024. https://doc.rust-lang.org/reference/src-ecmascript-2024.html

[31] Referencia Normativa src-unicode-15. https://doc.rust-lang.org/reference/src-unicode-15.html

[32] Referencia Normativa src-pierce-tapl. https://doc.rust-lang.org/reference/src-pierce-tapl.html

[33] Referencia Normativa src-knuth-taocp. https://doc.rust-lang.org/reference/src-knuth-taocp.html

[34] Referencia Normativa src-csharp-spec. https://doc.rust-lang.org/reference/src-csharp-spec.html

[35] Referencia Normativa src-kleppmann-ddia. https://doc.rust-lang.org/reference/src-kleppmann-ddia.html

[36] Referencia Normativa src-iso-c17. https://doc.rust-lang.org/reference/src-iso-c17.html